

## About this Book

This book is designed to be easy to follow. I avoid big leaps of thought. Any time a new concept is introduced, I explain it right away or tell you when I'll explain it. Core concepts are reinforced with exercises and multiple explanations so that you can check your assumptions and make sure you're following along.

I lead with examples. Instead of writing symbol soup, my goal is to make it easy for you to visualize these concepts. I also think we learn best by being able to recall something we already know, and examples make recall easier. So when you're trying to remember the difference between arrays and linked lists (explained in [chapter 2](#)), you can just think about getting seated for a movie. Also, at the risk of stating the obvious, I'm a visual learner. This book is chock-full of images.

The contents of the book are carefully curated. There's no need to write a book that covers every sorting algorithm—that's why we have Wikipedia and Khan Academy. All the algorithms I've included are practical. I've found them useful in my job as a software engineer, and they provide a good founda-

tion for more complex topics. Happy reading!

## Roadmap

The first three chapters of this book lay the foundations:

- **[Chapter 1](#)**—You’ll learn your first practical algorithm: binary search. You also learn to analyze the speed of an algorithm using Big O notation. Big O notation is used throughout the book to analyze how slow or fast an algorithm is.
- **[Chapter 2](#)**—You’ll learn about two fundamental data structures: arrays and linked lists. These data structures are used throughout the book, and they’re used to make more advanced data structures like hash tables (**[chapter 5](#)**).
- **[Chapter 3](#)**—You’ll learn about recursion, a handy technique used by many algorithms (such as quicksort, covered in **[chapter 4](#)**).

In my experience, Big O notation and recursion are challenging topics for beginners. So I’ve slowed down and spent extra time on these sections.

The rest of the book presents algorithms with broad applications:

- **Problem-solving techniques**—Covered in **[chapters 4, 8, and 9](#)**. If you come across a problem and

aren't sure how to solve it efficiently, try divide and conquer ([chapter 4](#)) or dynamic programming ([chapter 9](#)). Or you may realize there's no efficient solution, and get an approximate answer using a greedy algorithm instead ([chapter 8](#)).

- **Hash tables**— Covered in [chapter 5](#).

A hash table is a very useful data structure. It contains sets of key and value pairs, like a person's name and their email address, or a username and the associated password. It's hard to overstate hash tables' usefulness. When I want to solve a problem, the two plans of attack I start with are “Can I use a hash table?” and “Can I model this as a graph?”

- **Graph algorithms**— Covered in [chapters 6](#) and [7](#). Graphs are a way to model a network: a social network, or a network of roads, or neurons, or any other set of connections. Breadth-first search ([chapter 6](#)) and Dijkstra's algorithm ([chapter 7](#)) are ways to find the shortest distance between two points in a network: you can use this approach to calculate the degrees of separation between two people or the shortest route to a destination.

- **K-nearest neighbors (KNN)**— Covered in [chapter 10](#). This is a simple machine-learning algorithm. You can use KNN to build a recommendations system, an OCR engine, a

system to predict stock values—anything that involves predicting a value (“We think Adit will rate this movie 4 stars”) or classifying an object (“That letter is a Q”).

- **Next steps**— [Chapter 11](#) goes over 10 algorithms that would make good further reading.

## How to use this book

The order and contents of this book have been carefully designed. If you’re interested in a topic, feel free to jump ahead. Otherwise, read the chapters in order—they build on each other.

I strongly recommend executing the code for the examples yourself. I can’t stress this part enough. Just type out my code samples verbatim (or download them from

[www.manning.com/books/grokking-algorithms](http://www.manning.com/books/grokking-algorithms) or

[https://github.com/egonschiele/grokking\\_algorithms](https://github.com/egonschiele/grokking_algorithms)),

and execute them. You’ll retain a lot more if you do.

I also recommend doing the exercises in this book. The exercises are short—usually just a minute or two, sometimes 5 to 10 minutes. They will help you check your thinking, so you’ll know when you’re off track before you’ve gone too far.

## Who should read this book

This book is aimed at anyone who knows the basics of coding and wants to understand algorithms. Maybe you already have a coding problem and are trying to find an algorithmic solution. Or maybe you want to understand what algorithms are useful for. Here's a short, incomplete list of people who will probably find this book useful:

- Hobbyist coders
- Coding boot camp students
- Computer science grads looking for a refresher
- Physics/math/other grads who are interested in programming

## **Code conventions and downloads**

All the code examples in this book use Python 2.7. All code in the book is presented in a fixed-width font like this to separate it from ordinary text. Code annotations accompany some of the listings, highlighting important concepts.

You can download the code for the examples in the book from the publisher's website at

[www.manning.com/books/grokking-algorithms](http://www.manning.com/books/grokking-algorithms) or from

[https://github.com/egonschiele/grokking\\_algorithms](https://github.com/egonschiele/grokking_algorithms).

I believe you learn best when you really enjoy learning—so have fun, and run the code samples!

## About the author

*Aditya Bhargava* is a software engineer at Etsy, an online marketplace for hand-made goods. He has a master's degree in computer science from the University of Chicago. He also runs a popular illustrated tech blog at [adit.io](http://adit.io).

## Author Online

Purchase of *Grokking Algorithms* includes free access to a private web forum run by Manning Publications where you can make comments about the book, ask technical questions, and receive help from the author and from other users. To access the forum and subscribe to it, point your web browser to

[www.manning.com/books/grokking-algorithms](http://www.manning.com/books/grokking-algorithms). This page provides information on how to get on the forum once you are registered, what kind of help is available, and the rules of conduct on the forum.

Manning's commitment to our readers is to provide a venue where a meaningful dialog between individual readers and between readers and the author can take place. It isn't a commitment to any specific amount of participation on the part of the author, whose contribution to Author Online remains voluntary (and unpaid). We suggest you try asking the author some challenging

questions lest his interest stray! The Author Online forum and the archives of previous discussions will be accessible from the publisher's website as long as the book is in print.